Database Connections with FastAPI

SQL connections and NoSQL connections, explained deeply

1. What a database connection is
2. Connection strings, configuration and secrets
3. The layers: driver, ORM / ODM, your code
4. Connection pooling in depth
5. How FastAPI manages connections (lifecycle)
6. SQL connections: SQLite, PostgreSQL, MySQL
7. Sessions, transactions and error handling
8. NoSQL connections: MongoDB
9. NoSQL connections: Redis and others
10. Using several databases in one app
11. Docker, networking and production settings
12. Troubleshooting guide and cheat sheet

1. What a database connection is

A database runs as a separate program (a **server process**). Your FastAPI app is another program, the **client**. A **connection** is a live, authenticated communication channel between them, usually a TCP socket, over which the client sends commands (SQL text, MongoDB BSON messages, Redis commands) and receives results.



1.1 What happens when you connect

Step	What happens	Typical cost
1. DNS lookup	Host name (e.g. db.example.com) resolved to an IP	1-50 ms
2. TCP handshake	Three-way handshake to host:port	1 round trip
3. TLS handshake	Encrypt the channel, verify the server certificate (if enabled)	1-2 round trips
4. Authentication	Username/password (SCRAM, md5...), or token / certificate	1-2 round trips
5. Session setup	Select database, set encoding, time zone, search_path	1 round trip
6. Queries	Send request, receive rows, repeat	Per query
7. Close	Send terminate message, close socket	Small

Steps 1 to 5 can take **20-200 ms** and consume memory on the server (PostgreSQL starts one operating-system process per connection, roughly 5-10 MB each). That is why a web app must **reuse** connections instead of opening a new one per request. This is the single most important idea in this guide.

1.2 Connection vs session vs transaction

Term	Meaning
Connection	The network channel and the server-side process or thread behind it
Session (database)	Server-side state tied to one connection: settings, temp tables, current transaction
Session (SQLAlchemy)	Python object that tracks your ORM objects and borrows a connection when needed
Transaction	A group of statements on one connection that commit or roll back together
Pool	A cache of open connections that your code borrows and returns

1.3 Stateless HTTP and stateful connections

HTTP requests are independent and may run concurrently, but a database connection can run **only one query at a time** (for most drivers) and carries transaction state. So the rules are: never share one connection between two requests at the same time, always return it when finished, and keep transactions short.

2. Connection strings, configuration and secrets

2.1 Anatomy of a connection URL (DSN)

```
postgresql+asyncpg://app_user:s3cret@db.internal:5432/shop?sslmode=require
|          |        |        |       |           |    |    |
|          |        |        |       |           |    |    +-- options (query string)
|          |        |        |       |           |    +------- database name
|          |        |        |       |           +------------ port
|          |        |        |       +------------------------ host
|          |        |        +-------------------------------- password
|          |        +----------------------------------------- username
|          +-------------------------------------------------- driver (SQLAlchemy part)
+------------------------------------------------------------- dialect / database type
```

2.2 URLs for common databases

```
# SQL
sqlite:///./app.db                                        relative file
sqlite:////absolute/path/app.db                           absolute file (4 slashes)
sqlite+aiosqlite:///./app.db                              async SQLite
postgresql+psycopg2://user:pw@localhost:5432/shop         sync
postgresql+psycopg://user:pw@localhost:5432/shop          psycopg 3 (sync or async)
postgresql+asyncpg://user:pw@localhost:5432/shop          async
mysql+pymysql://user:pw@localhost:3306/shop?charset=utf8mb4
mysql+aiomysql://user:pw@localhost:3306/shop

# NoSQL
mongodb://user:pw@localhost:27017/shop?authSource=admin
mongodb://h1:27017,h2:27017,h3:27017/shop?replicaSet=rs0   replica set
mongodb+srv://user:pw@cluster0.abcde.mongodb.net/shop      Atlas (DNS SRV)
redis://:pw@localhost:6379/0                               db number 0
rediss://:pw@host:6380/0                                   rediss = TLS
```

Special characters: if the password contains @ : / ? # %, URL-encode it. In Python: from urllib.parse import quote_plus; quote_plus('p@ss/word') gives p%40ss%2Fword. Or build the URL with sqlalchemy.engine.URL.create(...) so no manual encoding is needed.

```
from sqlalchemy.engine import URL
url = URL.create(
    "postgresql+psycopg2",
    username="app_user", password="p@ss/word",
    host="localhost", port=5432, database="shop",
)
```

2.3 Configuration from environment variables

Never hard-code credentials. Read them from the environment so the same code runs on your laptop, in CI and in production with different values (the 12-factor app rule).

```
# pip install pydantic-settings
# app/core/config.py
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    # SQL
    database_url: str = "sqlite:///./app.db"
    db_pool_size: int = 10
    db_max_overflow: int = 20
    db_pool_timeout: int = 30
    db_echo: bool = False

    # NoSQL
    mongo_url: str = "mongodb://localhost:27017"
    mongo_db: str = "shop"
    redis_url: str = "redis://localhost:6379/0"

@lru_cache
```

```
def get_settings() -> Settings:
    return Settings()          # built once, reused everywhere

# .env   (add to .gitignore!)
DATABASE_URL=postgresql+psycopg2://app_user:s3cret@localhost:5432/shop
MONGO_URL=mongodb://localhost:27017
REDIS_URL=redis://localhost:6379/0
```

2.4 Secrets in production

- Use your platform's secret store: Docker/Kubernetes secrets, AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, GitHub Actions secrets.
- Do not print the URL in logs (it contains the password). Use engine.url.render_as_string(hide_password=True).
- Give the app a **least-privilege** database user (SELECT/INSERT/UPDATE/DELETE on its schema only). Use a separate, more powerful user for migrations.
- Rotate credentials periodically; design for connections that get refused after rotation (pool_recycle and retries).

3. The layers: driver, ORM / ODM, your code

```
Your FastAPI route functions
          |
ORM / ODM / query builder     SQLAlchemy, SQLModel   |  Beanie, MongoEngine
          |
Driver (database client)      psycopg2, asyncpg, PyMySQL | PyMongo, Motor | redis-py
          |
Wire protocol over TCP        PostgreSQL protocol | MySQL protocol | MongoDB wire protocol | RESP
          |
Database server
```

3.1 Choosing a layer

Approach	Pros	Cons	Use when
Raw driver (psycopg2, asyncpg, PyMongo)	Fastest, full control, no abstraction	Hand-written SQL, manual mapping, more code	Performance-critical paths, simple scripts
Query builder (SQLAlchemy Core)	Composable SQL in Python, injection-safe	No object mapping	Reporting, dynamic queries
ORM (SQLAlchemy ORM, SQLModel)	Classes instead of SQL, relationships, migrations	Learning curve, N+1 risk	Most CRUD apps
ODM (Beanie, MongoEngine)	Pydantic-like documents for MongoDB	Adds a dependency layer	Document-heavy MongoDB apps

3.2 Driver comparison

Database	Sync driver	Async driver	Notes
PostgreSQL	psycopg2-binary, psycopg 3	asyncpg, psycopg 3 (async)	asyncpg is fastest; psycopg 3 does both sync and async
MySQL / MariaDB	PyMySQL, mysqlclient	aiomysql, asyncmy	PyMySQL is pure Python, easy to install
SQLite	sqlite3 (standard library)	aiosqlite	File database; one writer at a time
SQL Server	pyodbc	aioodbc	Needs ODBC driver installed
MongoDB	PyMongo	Motor, PyMongo Async	Motor wraps PyMongo for asyncio
Redis	redis-py	redis.asyncio (in redis-py)	Same package, async namespace

3.3 Sync or async: how to decide

FastAPI runs def routes in a **threadpool** (40 threads by default) and async def routes directly on the **event loop**.

	Sync (def)	Async (async def)
Driver	psycopg2, PyMySQL, PyMongo	asyncpg, aiomysql, Motor
Concurrency model	One thread per in-flight request	One thread, many coroutines waiting on I/O
Max parallel DB calls	About the threadpool size	Limited by pool size, not threads
Danger	Pool smaller than threads causes waiting	A blocking call freezes the whole server
Simplicity	Simpler, great default	More moving parts, better at very high concurrency

Golden rule: never call a blocking function (psycopg2 query, time.sleep, requests.get) inside async def. It stops the event loop and every other request stalls. If you must, use def or await run_in_threadpool(func).

4. Connection pooling in depth

4.1 Why pool

```
WITHOUT a pool (per request):            WITH a pool:
 request -> connect (50 ms) -> query      request -> borrow (0.01 ms) -> query -> return
 request -> connect (50 ms) -> query      request -> borrow (0.01 ms) -> query -> return
 100 requests = 100 TCP+auth handshakes   100 requests reuse ~10 open connections
```

4.2 How a pool works

```
                +---------------------- Pool (size=3, max_overflow=2) --------------------+
 request A  --->  | [conn1 busy]  [conn2 idle]  [conn3 busy]   overflow: [ ] [ ]            |
 request B  --->  |   borrow conn2 -> run query -> RETURN conn2 to pool                     |
 request C  --->  |   all 3 busy -> open temporary overflow conn4 (closed when returned)    |
 request D,E --->  |   overflow also full -> WAIT up to pool_timeout, then raise error       |
                +-------------------------------------------------------------------------+
```

4.3 SQLAlchemy pool parameters

Parameter	Default	Meaning
pool_size	5	Connections kept open permanently
max_overflow	10	Extra connections allowed under burst; closed when returned
pool_timeout	30 s	How long to wait for a free connection before raising TimeoutError
pool_recycle	-1 (never)	Replace connections older than N seconds. Set below the server or firewall idle timeout (e.g. 1800)
pool_pre_ping	False	Send a lightweight test before handing out a connection; transparently replaces dead ones. Recommended True
pool_use_lifo	False	Reuse the most recently used connection so idle ones can expire
echo	False	Log all SQL (development only)

```
engine = create_engine(
    settings.database_url,
    pool_size=10, max_overflow=20, pool_timeout=30,
    pool_recycle=1800, pool_pre_ping=True,
)
```

4.4 Pool classes

Class	Used for
QueuePool	Default for most databases (sync). Thread-safe pool with a size limit
AsyncAdaptedQueuePool	Default for async engines
NullPool	No pooling; every checkout opens a new connection. Use with PgBouncer or serverless
StaticPool	One shared connection. Used for in-memory SQLite in tests
SingletonThreadPool	One connection per thread (SQLite memory)

4.5 Sizing the pool: do the math

The server has a hard limit on connections (PostgreSQL max_connections defaults to 100; MySQL max_connections to 151). Your app can open up to:

```
max connections = (pool_size + max_overflow) x workers x app instances

Example: pool_size=10, max_overflow=20  -> 30 per process
         4 Uvicorn/Gunicorn workers     -> 120 per instance
```

```
        3 instances (containers)       -> 360 connections   (but server allows 100!)

Fix: pool_size=5, max_overflow=5 -> 10 x 4 x 3 = 120 ... still too many.
     Use pool_size=3, max_overflow=2 -> 5 x 4 x 3 = 60. Keep ~20% headroom for admin/migrations.
```

- More connections is **not** faster. A database has limited CPU cores; a pool of 10-20 per instance is usually plenty.
- Each worker process has its **own** pool. Pools are never shared across processes.
- If you need thousands of client connections (many instances, serverless), put a pooler in front: **PgBouncer** (PostgreSQL), **ProxySQL** (MySQL), **Amazon RDS Proxy**.

4.6 PgBouncer in one minute

```
FastAPI x many  ->  PgBouncer (accepts 1000 client conns)  ->  PostgreSQL (uses 20 real conns)

pool modes:  session      one server conn per client session (safest)
             transaction  server conn lent only for one transaction (best reuse)
             statement    one statement at a time (no multi-statement transactions)

With asyncpg + transaction pooling, disable prepared-statement cache:
  create_async_engine(url, connect_args={"statement_cache_size": 0})
```

4.7 Connection leaks and stale connections

Symptom	Likely cause	Fix
QueuePool limit of size X overflow Y reached	Sessions never closed, or very slow queries hold connections	Always use a dependency with try/finally; add statement timeouts
server closed the connection unexpectedly	Firewall/load balancer or DB dropped an idle connection	pool_pre_ping=True, pool_recycle below idle timeout
too many connections (server error)	Pools x workers x instances exceed max_connections	Reduce pool, add PgBouncer, raise limit carefully
Requests hang, then time out	Pool exhausted, all connections busy	Find slow queries, increase pool modestly, check for leaks
MySQL server has gone away	wait_timeout expired	pool_recycle < wait_timeout

```
# Watch the pool at runtime
print(engine.pool.status())
# Pool size: 10  Connections in pool: 4  Current Overflow: 0  Current Checked out connections: 6
```

5. How FastAPI manages connections (lifecycle)

5.1 Application lifetime vs request lifetime

Scope	Created	Destroyed	Holds
Application	Once at startup (lifespan)	At shutdown	Engine / Mongo client / Redis client (and their pools)
Request	Per request (dependency)	After response	A SQLAlchemy Session (borrows one pooled connection briefly)

```
STARTUP   lifespan: create engine / clients (once)
   |
   |   request 1: Depends(get_db) -> new Session -> borrow conn -> query -> close -> return conn
   |   request 2: Depends(get_db) -> new Session -> borrow conn -> query -> close -> return conn
   |   (requests run concurrently, each with its own Session)
   |
SHUTDOWN  lifespan: engine.dispose() / client.close()  (close every pooled connection)
```

5.2 Dependency with yield (the standard pattern)

```
def get_db():
    db = SessionLocal()       # code BEFORE yield runs before the route
    try:
        yield db              # this value is injected into the route
    finally:
        db.close()            # code AFTER yield runs after the response is sent
```

- If the route raises an exception, the finally block still runs, so the connection is returned.
- FastAPI caches dependencies per request: if two dependencies both need get_db, they share one session.
- Do not store the session globally; a global session shared across requests causes data races.
- Do not rely on the request's session inside background tasks, which may run after the dependency has closed it; open a new session there.

5.3 Lifespan: create and close clients once

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---- startup ----
    app.state.engine = create_async_engine(settings.database_url, pool_pre_ping=True)
    app.state.session_factory = async_sessionmaker(app.state.engine, expire_on_commit=False)
    app.state.mongo = AsyncIOMotorClient(settings.mongo_url)
    app.state.redis = redis.from_url(settings.redis_url, decode_responses=True)
    yield
    # ---- shutdown (runs on SIGTERM / Ctrl+C) ----
    await app.state.redis.aclose()
    app.state.mongo.close()
    await app.state.engine.dispose()

app = FastAPI(lifespan=lifespan)
```

The old @app.on_event('startup') / ('shutdown') still works but is deprecated. Use lifespan; it keeps startup and shutdown logic in one place and shares variables between them.

5.4 Multiple workers

Running uvicorn app.main:app --workers 4 or Gunicorn with Uvicorn workers starts **4 separate processes**, each running lifespan and each creating its own engine and pool. Create engines **inside** the

worker (in lifespan or at import time of the worker), never before forking, because open sockets must not be shared across processes.

```
gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000
# workers rule of thumb: (2 x CPU cores) + 1, but cap by DB connection budget
```

6. SQL connections: SQLite, PostgreSQL, MySQL

6.1 Raw driver connection (no ORM)

Understanding the raw driver shows what SQLAlchemy does for you.

```
# PostgreSQL with psycopg2
import psycopg2
conn = psycopg2.connect(host="localhost", port=5432, dbname="shop",
                        user="app_user", password="s3cret", connect_timeout=5)
with conn:                                    # commits on success, rolls back on error
    with conn.cursor() as cur:
        cur.execute("SELECT id, name FROM users WHERE email = %s", ("a@x.com",))  # parameters!
        row = cur.fetchone()
conn.close()

# MySQL with PyMySQL
import pymysql
conn = pymysql.connect(host="localhost", user="app_user", password="s3cret",
                       database="shop", charset="utf8mb4",
                       cursorclass=pymysql.cursors.DictCursor, connect_timeout=5)
with conn.cursor() as cur:
    cur.execute("SELECT * FROM users WHERE id = %s", (1,))
    print(cur.fetchone())
conn.commit()
conn.close()

# SQLite (built in): a connection is simply opening a file
import sqlite3
conn = sqlite3.connect("app.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")      # off by default in SQLite!
```

6.2 Raw async pool with asyncpg

```
import asyncpg
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.pool = await asyncpg.create_pool(
        dsn="postgresql://app_user:s3cret@localhost:5432/shop",
        min_size=5, max_size=20, command_timeout=10)
    yield
    await app.state.pool.close()

app = FastAPI(lifespan=lifespan)

@app.get("/users/{uid}")
async def get_user(uid: int, request: Request):
    async with request.app.state.pool.acquire() as conn:      # borrow
        row = await conn.fetchrow("SELECT id, name FROM users WHERE id = $1", uid)
    return dict(row) if row else {"error": "not found"}        # returned to pool here
```

asyncpg uses $1, $2 placeholders; psycopg2 uses %s; SQLAlchemy text() uses :name; SQLite uses ?.

6.3 SQLAlchemy sync (recommended starting point)

```
# app/db/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from app.core.config import get_settings

settings = get_settings()

if settings.database_url.startswith("sqlite"):
    engine = create_engine(settings.database_url,
                           connect_args={"check_same_thread": False})
else:
    engine = create_engine(
        settings.database_url,
```

```
        pool_size=settings.db_pool_size,
        max_overflow=settings.db_max_overflow,
        pool_timeout=settings.db_pool_timeout,
        pool_recycle=1800,
        pool_pre_ping=True,
    )

SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

def get_db():
    with SessionLocal() as db:       # context manager closes the session
        yield db
```

The branch exists because SQLite (file-based) uses a different pool and needs check_same_thread=False, while server databases take the pool options. In real projects you normally pick one database and drop the branch.

6.4 SQLAlchemy async

```
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

engine = create_async_engine(
    "postgresql+asyncpg://app_user:s3cret@localhost:5432/shop",
    pool_size=10, max_overflow=10, pool_pre_ping=True, pool_recycle=1800,
)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session

@router.get("/users/{uid}")
async def read_user(uid: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == uid))
    user = result.scalar_one_or_none()
    if user is None:
        raise HTTPException(404, "User not found")
    return user
```

- expire_on_commit=False avoids reloading attributes after commit, which would trigger lazy IO in async code and raise MissingGreenlet.
- Use selectinload() / joinedload() for relationships in async code; lazy loading is not allowed.
- Create tables with async with engine.begin() as conn: await conn.run_sync(Base.metadata.create_all) (dev only; use Alembic in production).

6.5 PostgreSQL specifics

```
# SSL
postgresql+psycopg2://u:p@host/db?sslmode=require            (also verify-ca, verify-full)
# asyncpg uses connect_args:
create_async_engine(url, connect_args={"ssl": "require", "timeout": 10,
                                       "server_settings": {"application_name": "shop-api"}})
# Statement timeout so a runaway query cannot hold a connection forever
create_engine(url, connect_args={"options": "-c statement_timeout=15000"})   # 15 s
```

6.6 MySQL specifics

```
mysql+pymysql://u:p@host:3306/db?charset=utf8mb4          # utf8mb4 = full Unicode (emoji)
create_engine(url, pool_recycle=3600)     # keep below MySQL wait_timeout (default 8 h)
create_engine(url, connect_args={"ssl": {"ca": "/path/ca.pem"}})
```

6.7 SQLite specifics

- No server and no network: the **file** is the database. Great for learning, tests and small tools.

- check_same_thread=False is required because FastAPI may use the connection from a different thread than the one that created it.
- Only **one writer at a time**; concurrent writes wait or raise database is locked. Enable WAL mode (PRAGMA journal_mode=WAL) for better read/write concurrency.
- Foreign keys are off unless you run PRAGMA foreign_keys=ON on every new connection.
- In-memory test DB: sqlite:// with poolclass=StaticPool so all sessions see the same memory database.

```
from sqlalchemy import event
@event.listens_for(engine, "connect")
def set_sqlite_pragma(dbapi_conn, record):
    cur = dbapi_conn.cursor()
    cur.execute("PRAGMA foreign_keys=ON")
    cur.execute("PRAGMA journal_mode=WAL")
    cur.close()
```

6.8 SQLModel

```
from sqlmodel import SQLModel, Session, create_engine
engine = create_engine(settings.database_url, pool_pre_ping=True)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]    # from typing import Annotated

@app.get("/heroes")
def read_heroes(session: SessionDep):
    return session.exec(select(Hero)).all()
```

6.9 Read replicas (advanced)

```
write_engine = create_engine(PRIMARY_URL, pool_pre_ping=True)
read_engine  = create_engine(REPLICA_URL, pool_pre_ping=True)

WriteSession = sessionmaker(bind=write_engine)
ReadSession  = sessionmaker(bind=read_engine)

def get_read_db():
    with ReadSession() as db: yield db       # use for GET endpoints
def get_write_db():
    with WriteSession() as db: yield db      # use for POST/PUT/DELETE
# Beware replication lag: a read right after a write may not see the change yet.
```

7. Sessions, transactions and error handling

7.1 What a Session does

```
db.add(obj)        -> object is PENDING (nothing sent yet)
db.flush()         -> INSERT/UPDATE statements sent to DB inside the open transaction
db.commit()        -> flush + COMMIT (changes permanent), connection returned to the pool
db.rollback()      -> ROLLBACK (changes discarded)
db.refresh(obj)    -> re-SELECT the row to load server-generated values (id, defaults)
db.close()         -> release the connection; any uncommitted transaction is rolled back
```

The first statement starts a transaction automatically (SQLAlchemy 2.0 uses 'autobegin'). Until you **commit**, other connections cannot see your changes. If you forget commit(), your data silently disappears when the session closes.

7.2 One transaction per request

```
def get_db():
    db = SessionLocal()
    try:
        yield db
        db.commit()           # commit once, after the route finished without error
    except Exception:
        db.rollback()         # any exception -> undo everything
        raise
    finally:
        db.close()
```

This 'unit of work per request' pattern keeps routes short: they just db.add(...) and the dependency commits. The alternative is to call commit() explicitly in each route, which gives finer control. Choose one convention and use it consistently.

7.3 Handling database errors

Exception	Meaning	Suggested HTTP response
IntegrityError	Unique, foreign key or NOT NULL violation	409 Conflict or 422
NoResultFound	scalar_one() found nothing	404 Not Found
OperationalError	Cannot connect, server down, timeout, deadlock	503 Service Unavailable
TimeoutError (pool)	No free connection within pool_timeout	503 and an alert
DataError	Value too long or wrong type for the column	422
ProgrammingError	Bad SQL or missing table (a bug)	500, fix the code

```
from sqlalchemy.exc import IntegrityError, OperationalError

@app.exception_handler(IntegrityError)
async def integrity_handler(request, exc):
    return JSONResponse(status_code=409,
                        content={"detail": "Conflict: duplicate or invalid reference"})

@app.exception_handler(OperationalError)
async def operational_handler(request, exc):
    return JSONResponse(status_code=503, content={"detail": "Database temporarily unavailable"})
```

7.4 Retrying transient failures

```
# pip install tenacity
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type
from sqlalchemy.exc import OperationalError

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=0.2, max=2),
       retry=retry_if_exception_type(OperationalError), reraise=True)
```

```
def run_report(db: Session):
    return db.execute(text("SELECT ...")).all()
```

Only retry **idempotent** operations (reads, or writes you can safely repeat). Retrying a non-idempotent INSERT after a timeout can create duplicates because you do not know whether the first attempt succeeded.

7.5 Locking and isolation

```
# Row lock: other transactions wait until you commit
acct = db.execute(select(Account).where(Account.id == 1).with_for_update()).scalar_one()

# Skip locked rows (job queue pattern)
stmt = (select(Job).where(Job.status == "new")
        .with_for_update(skip_locked=True).limit(1))
job = db.execute(stmt).scalar_one_or_none()

# Per-engine isolation level
create_engine(url, isolation_level="REPEATABLE READ")
```

7.6 Health check endpoint

```
@app.get("/health/db")
def health_db(db: Session = Depends(get_db)):
    try:
        db.execute(text("SELECT 1"))
        return {"status": "ok"}
    except Exception:
        raise HTTPException(503, "database unavailable")
```

8. NoSQL connections: MongoDB

8.1 How the MongoDB client works

A **MongoClient** object is different from a SQL connection. It is a **thread-safe, long-lived manager** that owns a background monitor, discovers the cluster topology (primary and secondaries), and keeps its own connection pool per server. You create **one client per process** and share it; there is no per-request session to open or close for ordinary operations.

```
                        MongoClient (one per process)
                         |-- topology monitor (heartbeats every 10 s)
                         |-- pool to primary     (maxPoolSize connections)
                         |-- pool to secondary 1
                         '-- pool to secondary 2
db = client["shop"]            -> Database object (cheap, no network call)
col = db["users"]              -> Collection object (cheap, no network call)
await col.insert_one({...})    -> borrows a pooled connection for the call
```

8.2 Connection URI options

Option	Meaning	Typical value
maxPoolSize	Max connections per server	50 (default 100)
minPoolSize	Connections kept warm	0-10
maxIdleTimeMS	Close idle connections after	60000
serverSelectionTimeoutMS	How long to wait to find a suitable server before failing	5000 (default 30000)
connectTimeoutMS	TCP connect timeout	5000
socketTimeoutMS	Timeout for a single operation's socket	30000
retryWrites / retryReads	Auto-retry once on transient errors	true
w / journal	Write concern: acknowledge by how many nodes	w=majority
readPreference	primary, secondaryPreferred, nearest...	primary
replicaSet	Name of the replica set	rs0
authSource	Database holding the user's credentials	admin
tls	Use TLS	true on Atlas / production
appName	Shown in server logs to identify your app	shop-api

```
mongodb://app:pw@h1:27017,h2:27017,h3:27017/shop
    ?replicaSet=rs0&authSource=admin&tls=true
    &maxPoolSize=50&serverSelectionTimeoutMS=5000&retryWrites=true&w=majority&appName=shop-api
```

8.3 Motor (async) with FastAPI, full example

```
# pip install motor
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends, HTTPException, Request
from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
from pymongo.errors import DuplicateKeyError, ServerSelectionTimeoutError
from bson import ObjectId
from pydantic import BaseModel, EmailStr

@asynccontextmanager
async def lifespan(app: FastAPI):
    client = AsyncIOMotorClient(settings.mongo_url, maxPoolSize=50,
                                serverSelectionTimeoutMS=5000, appName="shop-api")
    await client.admin.command("ping")                      # fail fast if unreachable
    db = client[settings.mongo_db]
    await db.users.create_index("email", unique=True)       # idempotent
    app.state.mongo_client, app.state.mongo_db = client, db
```

```
    yield
    client.close()

app = FastAPI(lifespan=lifespan)

def get_mongo(request: Request) -> AsyncIOMotorDatabase:
    return request.app.state.mongo_db

class UserIn(BaseModel):
    name: str
    email: EmailStr

def serialize(doc: dict) -> dict:
    doc["id"] = str(doc.pop("_id"))
    return doc

@app.post("/users", status_code=201)
async def create_user(user: UserIn, db: AsyncIOMotorDatabase = Depends(get_mongo)):
    try:
        res = await db.users.insert_one(user.model_dump())
    except DuplicateKeyError:
        raise HTTPException(409, "Email already exists")
    return {"id": str(res.inserted_id)}

@app.get("/users/{uid}")
async def get_user(uid: str, db: AsyncIOMotorDatabase = Depends(get_mongo)):
    if not ObjectId.is_valid(uid):
        raise HTTPException(422, "Invalid id")
    doc = await db.users.find_one({"_id": ObjectId(uid)})
    if not doc:
        raise HTTPException(404, "Not found")
    return serialize(doc)

@app.get("/users")
async def list_users(skip: int = 0, limit: int = 20,
                     db: AsyncIOMotorDatabase = Depends(get_mongo)):
    cursor = db.users.find({}).sort("name", 1).skip(skip).limit(limit)
    return [serialize(d) async for d in cursor]
```

8.4 PyMongo (sync) with FastAPI

```
from pymongo import MongoClient

client = MongoClient(settings.mongo_url, maxPoolSize=50, serverSelectionTimeoutMS=5000)
db = client[settings.mongo_db]

@app.get("/users/{uid}")
def get_user(uid: str):                      # plain def -> runs in the threadpool, which is correct
    doc = db.users.find_one({"_id": ObjectId(uid)})
    ...
```

PyMongo's recent releases include a native async API (AsyncMongoClient) that is replacing Motor in the long term. Motor remains widely used and works well today; check MongoDB's current documentation when starting a new project.

8.5 Pydantic and ObjectId

```
from typing import Annotated
from pydantic import BaseModel, BeforeValidator, Field, ConfigDict

PyObjectId = Annotated[str, BeforeValidator(str)]       # convert ObjectId -> str

class UserOut(BaseModel):
    id: PyObjectId | None = Field(default=None, alias="_id")
    name: str
    email: str
    model_config = ConfigDict(populate_by_name=True)

@app.get("/users/{uid}", response_model=UserOut)
async def get_user(uid: str, db=Depends(get_mongo)):
    return await db.users.find_one({"_id": ObjectId(uid)})
```

8.6 Beanie ODM

```
# pip install beanie
from beanie import Document, Indexed, init_beanie

class User(Document):
    name: str
    email: Indexed(str, unique=True)
    class Settings:
        name = "users"                          # collection name

@asynccontextmanager
async def lifespan(app: FastAPI):
    client = AsyncIOMotorClient(settings.mongo_url)
    await init_beanie(database=client[settings.mongo_db], document_models=[User])
    yield
    client.close()

@app.post("/users")
async def create(u: User):
    await u.insert()
    return u

@app.get("/users/{email}")
async def by_email(email: str):
    return await User.find_one(User.email == email)
```

8.7 MongoDB transactions

```
async with await client.start_session() as s:            # requires a replica set
    async with s.start_transaction():
        await db.accounts.update_one({"_id": a}, {"$inc": {"bal": -100}}, session=s)
        await db.accounts.update_one({"_id": b}, {"$inc": {"bal": +100}}, session=s)
```

Single-document operations are always atomic, so design documents to keep related data together and you rarely need multi-document transactions. Transactions need a replica set (even a single-node one in development).

8.8 MongoDB security

- Enable authentication; never expose port 27017 to the internet (a classic cause of data leaks).
- Use TLS and a least-privilege user (readWrite on one database).
- Avoid NoSQL injection: never pass a raw request body as a filter. Validate with Pydantic types so {'$ne': None} cannot sneak in where a string is expected.

9. NoSQL connections: Redis and others

9.1 Redis connection model

Redis speaks a simple text protocol (RESP) over TCP. Redis itself is single-threaded for commands, so each command is very fast; the cost is mostly network latency. The Python client keeps a **connection pool**; one client object can be shared by the whole application.

```
import redis.asyncio as redis

pool = redis.ConnectionPool.from_url("redis://:pw@localhost:6379/0",
                                     max_connections=20, decode_responses=True,
                                     socket_connect_timeout=3, socket_timeout=3,
                                     health_check_interval=30)
r = redis.Redis(connection_pool=pool)

# or the shortcut
r = redis.from_url("redis://localhost:6379/0", decode_responses=True, max_connections=20)
```

Option	Meaning
decode_responses=True	Return str instead of bytes
max_connections	Pool limit; extra requests raise ConnectionError
socket_connect_timeout / socket_timeout	Fail fast instead of hanging forever
health_check_interval	Ping idle connections every N seconds to detect dead ones
retry / retry_on_timeout	Automatic retry policy for transient errors
ssl / rediss://	Encrypt the connection (managed Redis usually requires it)

9.2 Redis in FastAPI (lifespan and dependency)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = redis.from_url(settings.redis_url, decode_responses=True,
                                     max_connections=20, health_check_interval=30)
    await app.state.redis.ping()                    # fail fast at startup
    yield
    await app.state.redis.aclose()

def get_redis(request: Request) -> redis.Redis:
    return request.app.state.redis

@app.get("/counter")
async def counter(r: redis.Redis = Depends(get_redis)):
    return {"hits": await r.incr("hits")}
```

9.3 Handling Redis failures gracefully

A cache must never take the whole app down. If Redis is unreachable, fall back to the primary database and log the problem.

```
from redis.exceptions import RedisError

async def cached_user(uid: int, r, db):
    key = f"user:{uid}"
    try:
        hit = await r.get(key)
        if hit: return json.loads(hit)
    except RedisError:
        logger.warning("redis unavailable, falling back to DB")
    user = await db.get(User, uid)
    try:
        await r.set(key, json.dumps(to_dict(user)), ex=300)
    except RedisError:
```

```
        pass
    return to_dict(user)
```

9.4 Redis Sentinel and Cluster (high availability)

```
# Sentinel: automatic failover for a primary + replicas
from redis.asyncio.sentinel import Sentinel
sentinel = Sentinel([("s1", 26379), ("s2", 26379), ("s3", 26379)], socket_timeout=0.5)
master = sentinel.master_for("mymaster", decode_responses=True)

# Cluster: data sharded across many nodes
from redis.asyncio.cluster import RedisCluster
rc = RedisCluster(host="node1", port=6379, decode_responses=True)
```

9.5 Other NoSQL databases: connection snippets

```
# DynamoDB (AWS, HTTP API, no persistent connection) - boto3
import boto3
dynamodb = boto3.resource("dynamodb", region_name="ap-south-1")   # credentials from env/IAM role
table = dynamodb.Table("users")
table.put_item(Item={"id": "42", "name": "Asha"})
# boto3 is blocking: use plain def routes, or aioboto3 for async.

# Cassandra - cassandra-driver
from cassandra.cluster import Cluster
cluster = Cluster(["10.0.0.1", "10.0.0.2"], port=9042)   # contact points; rest auto-discovered
session = cluster.connect("shop")                          # one Session per application
rows = session.execute("SELECT * FROM users WHERE id=%s", (42,))

# Elasticsearch / OpenSearch
from elasticsearch import AsyncElasticsearch
es = AsyncElasticsearch("https://localhost:9200", api_key="...")
await es.search(index="products", query={"match": {"title": "phone"}})

# Neo4j (graph)
from neo4j import AsyncGraphDatabase
driver = AsyncGraphDatabase.driver("neo4j://localhost:7687", auth=("neo4j", "password"))
async with driver.session() as s:
    result = await s.run("MATCH (u:User)-[:FOLLOWS]->(f) WHERE u.id=$id RETURN f", id=42)
```

Database	Connection nature	Create client...
MongoDB	Persistent pooled sockets + topology monitor	once per process (lifespan)
Redis	Persistent pooled sockets	once per process (lifespan)
DynamoDB	Stateless HTTPS requests	once per process; credentials via IAM role
Cassandra	Persistent connections to many nodes	one Cluster and Session per process
Elasticsearch	HTTP connection pool	once per process
Neo4j	Bolt protocol pool	one Driver per process; short Sessions per unit of work

10. Using several databases in one app

Real applications often combine stores: SQL as the source of truth, Redis for caching, MongoDB for flexible documents (logs, events, catalogs). Each has its own client, pool and failure modes. Manage them all in one lifespan and expose each through a dependency.

```
app/
  core/config.py          settings: DATABASE_URL, MONGO_URL, REDIS_URL
  db/sql.py               engine, session factory, get_db
  db/mongo.py             client, get_mongo
  db/cache.py             redis, get_redis
  main.py                 lifespan wires everything together

# main.py
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.engine = create_async_engine(settings.database_url, pool_pre_ping=True)
    app.state.sessions = async_sessionmaker(app.state.engine, expire_on_commit=False)
    app.state.mongo = AsyncIOMotorClient(settings.mongo_url, serverSelectionTimeoutMS=5000)
    app.state.redis = redis.from_url(settings.redis_url, decode_responses=True)
    yield
    await app.state.redis.aclose()
    app.state.mongo.close()
    await app.state.engine.dispose()

async def get_sql(request: Request):
    async with request.app.state.sessions() as session:
        yield session

def get_mongo(request: Request):  return request.app.state.mongo[settings.mongo_db]
def get_redis(request: Request):  return request.app.state.redis

@app.post("/orders")
async def create_order(data: OrderIn,
                       sql: AsyncSession = Depends(get_sql),
                       mongo = Depends(get_mongo),
                       r = Depends(get_redis)):
    order = Order(**data.model_dump())
    sql.add(order)
    await sql.commit()                                          # 1. source of truth
    await mongo.order_events.insert_one({"order_id": order.id, "event": "created"})   # 2. audit log
    await r.delete(f"orders:user:{order.user_id}")              # 3. invalidate cache
    return {"id": order.id}
```

Consistency warning: there is no transaction across SQL, MongoDB and Redis. If step 2 fails after step 1 succeeded, the databases disagree. Commit the source of truth first, treat cache and logs as best-effort, or use an outbox table / message queue for reliable propagation.

10.1 Combined health endpoint

```
@app.get("/health")
async def health(sql=Depends(get_sql), mongo=Depends(get_mongo), r=Depends(get_redis)):
    checks = {}
    async def run(name, coro):
        try:    await coro; checks[name] = "ok"
        except Exception as e: checks[name] = f"error: {type(e).__name__}"
    await run("postgres", sql.execute(text("SELECT 1")))
    await run("mongo", mongo.command("ping"))
    await run("redis", r.ping())
    status = 200 if all(v == "ok" for v in checks.values()) else 503
    return JSONResponse(checks, status_code=status)
```

11. Docker, networking and production settings

11.1 Docker Compose with FastAPI and three databases

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql+asyncpg://app:secret@postgres:5432/shop     # host = service name!
      MONGO_URL: mongodb://mongo:27017
      REDIS_URL: redis://redis:6379/0
    depends_on:
      postgres: { condition: service_healthy }
  postgres:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: secret, POSTGRES_DB: shop }
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d shop"]
      interval: 5s
      retries: 10
  mongo:
    image: mongo:7
    volumes: [mongodata:/data/db]
  redis:
    image: redis:7
volumes: { pgdata: {}, mongodata: {} }
```

The most common Docker mistake: using localhost as the host inside a container. Inside the api container, localhost is the api container itself. Use the Compose **service name** (postgres, mongo, redis). depends_on only orders start-up; use a health check or connection retry so the app waits until the database accepts connections.

11.2 Waiting for the database at startup

```
import asyncio

async def wait_for_db(engine, retries=15, delay=2):
    for attempt in range(1, retries + 1):
        try:
            async with engine.connect() as conn:
                await conn.execute(text("SELECT 1"))
            return
        except Exception as e:
            logger.warning("DB not ready (%s/%s): %s", attempt, retries, type(e).__name__)
            await asyncio.sleep(delay)
    raise RuntimeError("Database never became available")
```

11.3 Timeouts: the settings that save you in production

Layer	Setting	Why
Connect	connect_timeout / connectTimeoutMS / socket_connect_timeout = 3-5 s	Do not hang forever if the host is unreachable
Query	statement_timeout (Postgres), max_execution_time (MySQL), maxTimeMS (Mongo)	Stop runaway queries from holding connections
Pool	pool_timeout = 10-30 s	Fail fast when saturated instead of piling up requests
Idle	pool_recycle, maxIdleTimeMS, health_check_interval	Avoid stale connections killed by firewalls
HTTP	Reverse proxy / client timeouts	Requests should not outlive the user's patience

11.4 Production checklist

- Engines and clients are created **once per process** in lifespan and closed on shutdown.
- Pool math done: (pool_size + max_overflow) x workers x instances below the database limit.
- pool_pre_ping=True and pool_recycle set for SQL; health_check_interval for Redis.
- Timeouts configured on connect, query and pool.
- TLS enabled to the database; credentials come from a secret store; least-privilege users.
- Database ports are **not** exposed publicly; use private networking / security groups.
- Migrations (Alembic) run as a deploy step, not inside app start-up with many workers racing.
- Health endpoint checks every dependency; orchestrator uses it for readiness.
- Slow query log, pool usage, connection counts and error rates are monitored.
- Backups are automated and a restore has been tested.
- Graceful shutdown: on SIGTERM stop accepting requests, finish in-flight ones, then close pools.

12. Troubleshooting guide and cheat sheet

12.1 Error messages and fixes

Error	Cause	Fix
Connection refused	Server not running, wrong host or port, or firewall	Check the service is up; check host/port; in Docker use service name, not localhost
password authentication failed for user	Wrong credentials, or special characters not URL-encoded	Verify user/password; quote_plus the password; check pg_hba.conf
database "x" does not exist	Database not created	CREATE DATABASE x; or fix the name in the URL
could not translate host name	DNS failure / typo in host	Check the host; check container network
no pg_hba.conf entry for host	PostgreSQL does not allow this client or lacks SSL	Add entry, or use sslmode=require
SSL connection is required / certificate verify failed	TLS settings mismatch	Set sslmode / ca file correctly
QueuePool limit ... reached	Pool exhausted or leaked sessions	Close sessions (dependency), find slow queries, tune pool
too many connections / too many clients	Total connections exceed server limit	Lower pool sizes, add PgBouncer
server closed the connection unexpectedly / gone away	Idle timeout or DB restart	pool_pre_ping, pool_recycle
MissingGreenlet (async)	Lazy-loaded attribute touched outside await	selectinload/joinedload; expire_on_commit=False
SQLite: database is locked	Concurrent writers	Short transactions, WAL mode, or use PostgreSQL
Object of type ObjectId is not JSON serializable	Returning raw Mongo documents	Convert _id to str, or use a Pydantic model
ServerSelectionTimeoutError (Mongo)	Cannot reach a Mongo node	Check URI, network, replica set name, IP allow list (Atlas)
Redis ConnectionError / TimeoutError	Redis down or pool exhausted	Check host; raise max_connections; handle errors with fallback
Greenlet / 'attached to a different loop'	Client created outside the running event loop	Create async clients inside lifespan

12.2 Quick connection test script

```
# test_connections.py - run: python test_connections.py
import asyncio, os
from sqlalchemy import text
from sqlalchemy.ext.asyncio import create_async_engine
from motor.motor_asyncio import AsyncIOMotorClient
import redis.asyncio as redis

async def main():
    eng = create_async_engine(os.environ["DATABASE_URL"])
    async with eng.connect() as c:
        print("SQL  :", (await c.execute(text("SELECT version()"))).scalar())
    await eng.dispose()

    m = AsyncIOMotorClient(os.environ["MONGO_URL"], serverSelectionTimeoutMS=3000)
    print("Mongo:", (await m.admin.command("ping")))
    m.close()
```

```
    r = redis.from_url(os.environ["REDIS_URL"])
    print("Redis:", await r.ping())
    await r.aclose()

asyncio.run(main())
```

12.3 One-page cheat sheet

Topic	Remember
Connection cost	Opening is slow (TCP + TLS + auth). Always reuse via a pool.
Where to create	Engine / Mongo client / Redis client: once per process, in lifespan. Session: once per request.
Closing	yield dependency with try/finally. Dispose engines and close clients on shutdown.
Sync vs async	def + psycopg2/PyMongo, or async def + asyncpg/Motor. Never block inside async def.
Pool size	(pool_size + max_overflow) x workers x instances < server max_connections.
Stale connections	pool_pre_ping=True, pool_recycle < idle timeout.
Transactions	commit() or it vanishes; rollback() on error; keep short; no slow HTTP calls inside.
Errors	IntegrityError 409, NoResult 404, OperationalError 503.
MongoDB	One MongoClient per process; serverSelectionTimeoutMS; convert ObjectId.
Redis	Pool with timeouts; treat as best-effort cache with DB fallback.
Docker	Host = service name. Wait for the DB (healthcheck or retry).
Security	Env vars/secret store, TLS, least privilege, parameterised queries, no public DB ports.

Practice idea: run the Docker Compose file from section 11, then build the multi-database /orders endpoint from section 10 and break things on purpose (stop Redis, lower the pool to 1, add a slow query) to see each failure mode first-hand.